

Tema 6: Funciones

Oscar Perpiñán Lamigueiro - David Álvarez

- 1 Definición de función
- 2 Ámbito de una variable
- 3 Funciones que llaman a otras funciones
- 4 Funciones y otros tipos de datos
- 5 Funciones en librerías

¿Qué es una función?

Una función es un bloque de código que realiza una tarea determinada a partir de unos datos.

Ventajas

- Permiten **programación estructurada y abstracta**, sin necesidad de conocer el detalle de la implementación de una tarea concreta.
- Mejoran la **legibilidad** del código.
- Facilitan el **mantenimiento** del programa.
- Permiten **reutilizar código** de manera eficiente.

¿Cómo se declara una función?

Prototipo de una función:

- 1 Tipo de valor que devuelve (int, void, ...)
- 2 Nombre de la función (debe ser un identificador válido y **útil**).
- 3 Lista de argumentos que emplea, por tipo y nombre (puede estar vacía).

```
tipo nombre_funcion(tipo1 arg1, tipo2 arg2, ...);
```

Ejemplos

```
void printHello(int veces);
```

```
float areaTriangulo(float b, float h);
```

¿Cómo se define una función? Ejemplo 1.

```
// Definicion de la funcion printHello

// No devuelve nada (void)

// Necesita un argumento: un número entero que indica las veces que se imprime //
// la frase (será entero (int) )

void printHello(int veces)
{
    int i;
    for (i = 1; i <= veces; i++)
        printf("Hello World!\n");
}
```

¿Cómo se define una función? Ejemplo 2.

```
// Definicion de la funcion areaTriangulo

// Devuelve un real (float)

// Necesita dos argumentos, b y h, reales.

float areaTriangulo(float b, float h)
{
    float area;

    area = b * h / 2.0;

    return area;
}
```

En un programa: se declara, define y usa cada función.

```
#include <stdio.h>

// se declara el prototipo de la función
void printHello(int veces);

int main()
{
    // se usa la función
    printHello(3);

    return 0;
}
```

```
// se define la función
void printHello(int veces)
{
    int i = 0;
    for(i=0; i<veces;i++)
    {
        printf("Buenos días, Como estan  
los maquinas? Lo primero de  
todo.\n");
    }
}
```

Sigue el código a la derecha...

- 1 Definición de función
- 2 **Ámbito de una variable**
- 3 Funciones que llaman a otras funciones
- 4 Funciones y otros tipos de datos
- 5 Funciones en librerías

Variables globales - prohibido usarlas

```
#include <stdio.h>

int globalVar = 3; //Variable global

void funcion_ejemplo(void);

int main(){
    printf("main (1):\t globalVar es %d.\n", globalVar);
    funcion_ejemplo();
    globalVar = globalVar*2;
    printf("main (2):\t globalVar es %d.\n", globalVar);
    return 0;
}

void funcion_ejemplo(void){
    globalVar = globalVar + 1;
    printf("funcion:\t globalVar es %d.\n", globalVar);
}
```

Variables locales

```
#include <stdio.h>

void funcion_ejemplo(void);

int main()
{
    int x = 1; // variable local en main
    printf("main (1):\t x vale %d.\n", x);
    funcion_ejemplo();
    printf("main (2):\t x vale %d.\n", x);
    return 0;
}

void funcion_ejemplo(void)
{
    int x = 2; // variable local en funcion_ejemplo
    printf("funcion:\t x vale %d.\n", x);
}
```

- 1 Definición de función
- 2 Ámbito de una variable
- 3 Funciones que llaman a otras funciones**
- 4 Funciones y otros tipos de datos
- 5 Funciones en librerías

Ejemplo

```
#include <stdio.h>
#define PI 3.141592

float eleva3(float x);
float volEsfera(float r);

int main(){
    float radio, vol;
    scanf("%f", &radio);
    vol = volEsfera(radio);
    printf("El volumen es %f", vol);
    return 0;
}

float volEsfera(float r){ //Usa eleva3
    return 4.0/3.0 * PI * eleva3(r);
}

float eleva3(float x){
    return x * x * x;
}
```

Funciones recursivas

```
#include <stdio.h>

int fact(int n);

int main(){
    int x;
    printf("Indica un número:\n");
    scanf("%d", &x);
    printf("El factorial de %d es %d\n", x, fact(x));
    return 0;
}

int fact(int n){
    int res;
    if (n > 1) // Incluye llamada a si misma
        res = n * fact(n - 1);
    else
        res = 1;
    return res;
}
```

Ejemplos de funciones

Genere un programa que incluya la declaración, uso y definición de las siguientes funciones:

- Área del cuadrado.
- Área del triángulo.

Dada la relación entre grados centígrados y Fahrenhaeit: $T_F = 9/5 * T_C + 32$

- Conversión de grados centígrados a Fahrenhaeit, y viceversa.
- Cálculo del máximo de tres valores de tipo entero. Usa una función max3 que admite como parámetros tres números enteros y entrega el valor del mayor. Esta función usará a su vez a otra función max2 que calcula el máximo entre 2 valores que se pasan como parámetros a la función.

- 1 Definición de función
- 2 Ámbito de una variable
- 3 Funciones que llaman a otras funciones
- 4 Funciones y otros tipos de datos**
- 5 Funciones en librerías

Vectores y cadenas de caracteres

- Los prototipos de funciones pueden incluir vectores o cadenas.
- En el caso de vectores, hay que incluir en el prototipo el tamaño.
- Dentro de la función, **¡se puede modificar el contenido del vector/cadena!**

Ejemplos prototipos

```
int max(int num[], int tam);  
void ordenar(int num[], int tam);  
int cuenta_vocales(char frase[]);  
void paso_a_mayusculas(char frase[]);
```


Ejemplo cadenas

```
#include <stdio.h>

void paso_a_mayusculas(char frase[]);

int main() {
    char ejemplo[30] = "Suerte en el examen.";
    printf("La frase: %s en mayusculas, \n\n", ejemplo);
    paso_a_mayusculas(ejemplo);
    printf("%s", ejemplo);
    return 0;
}

void paso_a_mayusculas(char frase[]) {
    int i = 0;
    while(frase[i] != '\0') {
        if(frase[i] > 96 && frase[i] < 123)
            frase[i] = frase[i] - ('a'-'A');
        i++;
    }
}
```

Ejemplo cadenas

```
#include <stdio.h>

void paso_a_mayusculas(char frase[]);

int main() {
    char ejemplo[30] = "Suerte en el examen.";
    printf("La frase: %s en mayusculas, \n\n", ejemplo);
    paso_a_mayusculas(ejemplo);
    printf("%s", ejemplo);
    return 0;
}

void paso_a_mayusculas(char frase[]) {
    int i = 0;
    while(frase[i] != '\0') {
        if(frase[i] > 96 && frase[i] < 123)
            frase[i] = frase[i] - ('a'-'A');
        i++;
    }
}
```

Estructuras

- Se usan exactamente igual que los datos comunes.
- Se pasa a la función el valor completo de la estructura.
- No pueden modificarse esos valores.
- Puede usarse como tipo de dato devuelto por la función.

Ejemplos prototipos con estructuras

```
typedef struct
{
    float real, imaginaria;
} complejo;

void imprimir(complejo c);
float modulo(complejo c);
complejo multiplica(complejo a,complejo b);
```

- 1 Definición de función
- 2 Ámbito de una variable
- 3 Funciones que llaman a otras funciones
- 4 Funciones y otros tipos de datos
- 5 Funciones en librerías

Motivación y uso

Motivación

Para poder reutilizar las funciones definidas es conveniente alojarlas en un fichero (o colección de ficheros) que puedan ser incluidos en otros proyectos.

Uso

- Debe existir un (o varios) fichero(s) .h (cabecera) y un fichero .c (código fuente, implementación de las funciones).
- Se debe usar `#include "nombre_lib.h"` al comienzo del programa.
- Hay que compilar conjuntamente (*en un proyecto*).

Ejemplo (1)

Fichero myLib.h (cabecera)

```
#define PI 3.141592

float eleva3(float x);
float volEsfera(float r);
```

Fichero myLib.c (código fuente)

```
#include "myLib.h"

float volEsfera(float r){
    return 4.0/3.0 * PI * eleva3(r);
}

float eleva3(float x){
    return x * x * x;
}
```

Ejemplo (2)

Programa principal

```
#include <stdio.h>
// Directiva para incluir la librería local
#include "myLib.h"

int main()
{
    float radio, vol;
    scanf("%f", &radio);
    vol = volEsfera(radio);
    printf("El volumen es %f", vol);
    return 0;
}
```